

Assignment 1 - R3: Scaling Study and Report

Joshua Ganschow

CSC718

Prof. Abid Mehmood

Run Conditions

The following conditions are held constant through the runs:

- Number of jobs: 32
- Job type and workload
 - o A simple job with enough computation required to make the single-worker run take a few seconds
- Input data
 - o The only thing that changes is the number of workers
 - o The bound on the queue stays as 32, allowing all jobs to be queued. This makes the bottleneck solely the speed at which jobs are being completed, negligible additional overhead for waiting for queue operations.
- Timing method
 - o Using `clock_gettime` from `time.h`
- Computing environment
 - o Ryzen 7 7700X (8 core) running Fedora Linux
 - o Compiled using the GNU C Compiler

Each scenario gets run 10 times and averaged to get the clock time. The first three runs of the program are included in each screenshot.

Results Table

Number of workers	Execution time (s)	Speedup (T(1)/T(N))	Screenshot number
1	2.06359	1	1
2	1.068267	1.93172	2
4	0.546278	3.77755	3
8	0.289393	7.13075	4

Execution Evidence

- ```
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$./r2 1 32
Number of worker threads: 1
Number of jobs processed: 32
Total elapsed time: 2.080047
Jobs processed per second: 15.384265
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$./r2 1 32
Number of worker threads: 1
Number of jobs processed: 32
Total elapsed time: 2.072441
Jobs processed per second: 15.440726
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$./r2 1 32
Number of worker threads: 1
Number of jobs processed: 32
Total elapsed time: 2.073430
```
- ```
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$ ./r2 2 32
Number of worker threads: 2
Number of jobs processed: 32
Total elapsed time: 1.073156
Jobs processed per second: 29.818588
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$ ./r2 2 32
Number of worker threads: 2
Number of jobs processed: 32
Total elapsed time: 1.075981
Jobs processed per second: 29.740298
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$ ./r2 2 32
Number of worker threads: 2
Number of jobs processed: 32
Total elapsed time: 1.063000
Jobs processed per second: 30.103482
```
- ```
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$./r2 4 32
Number of worker threads: 4
Number of jobs processed: 32
Total elapsed time: 0.544740
Jobs processed per second: 58.743584
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$./r2 4 32
Number of worker threads: 4
Number of jobs processed: 32
Total elapsed time: 0.556086
Jobs processed per second: 57.545016
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$./r2 4 32
Number of worker threads: 4
Number of jobs processed: 32
Total elapsed time: 0.549825
Jobs processed per second: 58.200387
```
- ```
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$ ./r2 8 32
Number of worker threads: 8
Number of jobs processed: 32
Total elapsed time: 0.278567
Jobs processed per second: 114.873487
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$ ./r2 8 32
Number of worker threads: 8
Number of jobs processed: 32
Total elapsed time: 0.314877
Jobs processed per second: 101.627024
joshua@fedora:~/school/Fall 2026/CSC718/content/module 2 - multithreaded programming/assignment1$ ./r2 8 32
Number of worker threads: 8
Number of jobs processed: 32
Total elapsed time: 0.284346
Jobs processed per second: 112.538762
```

Technical Discussion

Seeing the speedups, we get nearly ideal numbers. The version with two workers takes roughly $\frac{1}{2}$ as much time, with four workers we see a $\sim 3.8x$ speedup, with eight we see $7.1x$ speedup. We can account for it not being exactly ideal because there is inherent overhead with enqueueing and dequeuing. Namely, shared variables are used among all workers for tracking and synchronizing. This requires a protected (critical) section surrounded with an operation to unlock and re-lock a mutex.

Additionally, due to the nature of the testing, identical jobs are enqueued for each of the M jobs. This means that it is more likely that multiple threads are reaching the critical section at nearly the exact same time. A real-world scenario with varying job workloads will not have the same problem, in general. This becomes clearer when we observe that, as the program is run with more threads, the returns slowly diminish (2 threads are almost exactly $2x$ faster, 4 is slightly further from $4x$, and 8 is even further from $8x$). This gives more reason to believe that the competition for the mutex and access to the queue is causing a bottleneck.